Week 1 - Domain 1: SDLC Automation - AWS CodeArtifact

Study week: Oct 19 - Oct 25, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

Covers: domains, repositories, upstreams, origin controls, npm / pip / Maven, cross-account, VPC endpoints

Part 3: AWS CodeArtifact

3.1 What CodeArtifact is

- **A managed, private package repository** - a private npm, PyPI, or Maven Central inside your AWS account. Replaces self-hosted Nexus, Artifactory, or verdaccio.
- **Two jobs:**
  1. **Stores your company's own packages** (e.g. `@mycorp/utils`, `mycorp-common.jar`).
  2. **Proxies and caches public packages** from npmjs, PyPI, Maven Central - builds keep working if the public site is down, and you control what is allowed.
- **Formats:** npm (yarn, pnpm), Python (pip, twine), Maven / Gradle, NuGet, Swift, Ruby, Cargo, and **generic** packages.
- Encrypted with KMS; access by IAM **and resource policies** (unlike CodeCommit); CloudTrail logging; EventBridge events.

3.2 Domains

- A **domain** is the **top-level container**. Every repository lives inside one domain.
- **Package files are stored once per domain** - the same file in 10 repositories is paid for once.
- **One KMS key per domain**, chosen at creation and **cannot be changed afterwards** (common exam trap).
- A **domain policy** (resource-based) controls **cross-account access**, especially who can get an authorization token.
- The **domain owner account** pays for storage.
- **Best practice:** one domain for the whole organization in a shared "artifacts" / "tools" account, separate repositories per team or environment.

```
Domain: mycorp  (owned by Tools account, KMS key: mycorp-artifacts-key)
 |-- npm-store      (external connection: public:npmjs)
 |-- pypi-store     (external connection: public:pypi)
 |-- maven-store    (external connection: public:maven-central)
 |-- team-a-repo    (upstreams: npm-store, pypi-store)
 '-- release-repo   (approved and promoted packages only)
```

3.3 Repositories

- A **repository** holds **package versions**; one repository can hold several formats.
- A **repository policy** (resource-based) controls who can read or publish, including other accounts.
- An **external connection** links a repository to a public site: `public:npmjs`, `public:pypi`, `public:maven-central`, `public:nuget-org`, `public:crates-io`.
- **Only one external connection per repository** - create one "store" repository per public source.
- **Package version status:** Published, Unfinished, Unlisted, Archived, Disposed. Change status to stop a bad version being used.
- **Promotion:** `copy-package-versions` copies a version from `dev-repo` to `release-repo` after tests pass (often a CodePipeline step).

3.4 Upstream repositories

- A repository can list **up to 10 upstream repositories**, searched **in the order listed**, up to a repository with an external connection.

- A version fetched through upstreams is **kept (copied) in the downstream repository** you requested from, and in the repositories in between. It stays available even if removed from the public site.

```
Developer / CodeBuild
        |   npm install lodash
        v
  team-a-repo  --(upstream)-->  npm-store  --(external connection)-->  registry.npmjs.org
  (lodash kept here)            (lodash kept here)
```

- **Why chain repositories:** one place to control and audit public packages; teams get internal and public packages from **one endpoint**.

Package origin controls (dependency confusion protection):

- **Publish:** ALLOW / BLOCK - can versions be published directly to this repository?
- **Upstream:** ALLOW / BLOCK - can versions come in from upstream repositories?

How the package first arrived	Publish	Upstream	Effect
Your company published it first	ALLOW	BLOCK	Blocks public versions
It first came from upstream	BLOCK	ALLOW	You can't publish versions under that name

- Stops an attacker publishing a public package with the **same name as your internal package** and a higher version ("dependency confusion" / "dependency substitution").
- **Exam keywords:** "malicious public package with same name as internal package" = **package origin controls**.

3.5 Authentication and permissions

- You need a **temporary authorization token** - valid **12 hours by default** (15 minutes to 12 hours).

```
aws codeartifact get-authorization-token --domain mycorp --domain-owner 111122223333 --query
    authorizationToken --output text
```

- `aws codeartifact login` gets a token **and** configures the tool (npm, pip, twine only):

```
aws codeartifact login --tool npm --domain mycorp --domain-owner 111122223333 --repository team-a-repo
```

To do this	You need
Get a token	codeartifact:GetAuthorizationToken AND sts:GetServiceBearerToken (condition sts:AWSServiceName = codeartifact.amazonaws.com)
Download packages	codeartifact:ReadFromRepository, codeartifact:GetRepositoryEndpoint
Publish packages	codeartifact:PublishPackageVersion, codeartifact:PutPackageMetadata

Exam trap: "not authorized to get token" despite CodeArtifact permissions = add `sts:GetServiceBearerToken`.

3.6 Setting up npm, pip, and Maven

npm - writes the registry URL and token into `~/.npmrc` (add `--namespace mycorp` to scope only `@mycorp/*`):

```
aws codeartifact login --tool npm --domain mycorp --domain-owner 111122223333 --repository team-a-repo
npm install
npm publish
```

pip and twine - pip login sets `index-url` in `pip.conf`; twine login sets up `~/.pypirc`:

```
aws codeartifact login --tool pip --domain mycorp --domain-owner 111122223333 --repository team-a-repo
pip install -r requirements.txt

aws codeartifact login --tool twine --domain mycorp --domain-owner 111122223333 --repository team-a-repo
twine upload --repository codeartifact dist/*
```

Maven and Gradle - `aws codeartifact login` does **not** support them; set the token yourself.

```
export CODEARTIFACT_AUTH_TOKEN=$(aws codeartifact get-authorization-token --domain mycorp --domain-owner
    111122223333 --query authorizationToken --output text)
```

```
<!-- settings.xml -->
<servers>
  <server>
    <id>mycorp-team-a-repo</id>
    <username>aws</username>
    <password>${env.CODEARTIFACT_AUTH_TOKEN}</password>
  </server>
</servers>

<!-- pom.xml (URL from: get-repository-endpoint --format maven) -->
<repositories>
  <repository>
    <id>mycorp-team-a-repo</id>
    <url>https://mycorp-111122223333.d.codeartifact.us-east-1.amazonaws.com/maven/team-a-repo/</url>
  </repository>
</repositories>
<distributionManagement>   <!-- for mvn deploy (publishing) -->
  <repository>
    <id>mycorp-team-a-repo</id>
    <url>https://mycorp-111122223333.d.codeartifact.us-east-1.amazonaws.com/maven/team-a-repo/</url>
  </repository>
</distributionManagement>
```

- **Gradle:** a `maven { url ...; credentials { username "aws"; password System.env.CODEARTIFACT_AUTH_TOKEN } }` block.

In CodeBuild (service role needs the permissions in 3.5):

```
phases:
  pre_build:
    commands:
      - aws codeartifact login --tool pip --domain mycorp --domain-owner 111122223333 --repository
    team-a-repo
  build:
    commands:
      - pip install -r requirements.txt
      - python -m build
      - aws codeartifact login --tool twine --domain mycorp --domain-owner 111122223333 --repository
    team-a-repo
      - twine upload --repository codeartifact dist/*
```

3.7 Cross-account access

Scenario: domain and repository in the **Tools account**, builds in the **Dev account**.

1. **Domain policy** (Tools): allow the Dev role `codeartifact:GetAuthorizationToken`.
2. **Repository policy** (Tools): allow the Dev role `codeartifact:ReadFromRepository` (and `PublishPackageVersion` if it publishes).
3. **IAM policy** (Dev): same permissions plus `sts:GetServiceBearerToken`.
4. **KMS:** if the domain uses a customer-managed key, the key policy must allow the other account.

Both sides must allow access: **resource policy** in Tools **and IAM policy** in Dev.

3.8 Using CodeArtifact from a private VPC (no internet)

- Builds in private subnets can use CodeArtifact **without a NAT gateway** - CodeArtifact fetches public packages **on your behalf**.

Endpoint	Type	Notes
com.amazonaws.<region>.codeartifact.api	Interface	API calls and getting tokens
com.amazonaws.<region>.codeartifact.repositories	Interface	npm / pip / maven traffic; turn on private DNS
S3	Gateway	Package files are stored in S3 - allow the CodeArtifact asset bucket

Endpoint	Type	Notes
STS	Interface	Only if the build gets credentials through STS

Exam scenario: "Builds in a private subnet with no internet must install public npm packages, without a NAT gateway" = **CodeArtifact external connection + CodeArtifact VPC endpoints + S3 gateway endpoint**.

3.9 Events and automation

- CodeArtifact sends a **package version state change** event to **EventBridge** when a version is published, changed, or deleted.
- "When a new version of the shared library is published, rebuild dependent services" = **EventBridge rule -> CodePipeline or CodeBuild**.

3.10 Exam traps: quick table

Scenario	Answer
Replace self-hosted Nexus / Artifactory with less work	CodeArtifact
Cache public npm / PyPI so builds survive a public outage	Repository with an external connection, used as an upstream
One endpoint for internal and public packages	Team repository with upstreams to store repositories
Dependency confusion / malicious same-name public package	Package origin controls (block upstream for internal names)
Store packages once across many repositories	They share one domain
Change the domain's KMS key	Not possible - create a new domain and migrate
Share packages with other accounts	Domain policy + repository policy + IAM in the other account
Private subnet needs public packages without NAT	CodeArtifact + VPC endpoints (api, repositories) + S3 gateway endpoint
Token request fails despite CodeArtifact permissions	Add sts:GetServiceBearerToken
Promote a tested package from dev to release	copy-package-versions
Rebuild dependents when a library is published	EventBridge rule -> CodePipeline / CodeBuild
Connect one repository to both npmjs and PyPI	Not possible - one external connection per repository; use store repos as upstreams

3.11 Hands-on lab (about 1.5 hours)

1. Create the domain `dop-lab` with the default `aws/codeartifact` key.
2. Create `pypi-store` with an external connection to `public:pypi`.
3. Create `team-repo` with `pypi-store` as its upstream.
4. Run `aws codeartifact login --tool pip ...` for `team-repo`, then `pip install requests`. Check that `requests` appears in **both** `team-repo` and `pypi-store`.
5. Build a tiny Python package `dop-lab-utils` and publish it with twine to `team-repo`.
6. Check the package origin controls for `dop-lab-utils`: publish ALLOW, upstream BLOCK.
7. Update your CodeBuild project to install dependencies through CodeArtifact (add read permissions and `sts:GetServiceBearerToken` to the service role).
8. Create an EventBridge rule that emails you (SNS) when a package version is published; publish version 0.2.
9. Clean up: delete the repositories, then the domain.

3.12 Check yourself

1. Your team needs both PyPI and npm public packages through one repository endpoint. How do you set that up?

2. An attacker publishes `mycorp-auth` 99.0 on public PyPI, with the same name as your internal package. What stops builds pulling it?
3. CodeBuild in private subnets with no NAT gateway must install from CodeArtifact. Which endpoints are required?
4. What two permissions does a role need just to **get a token**?
5. The domain uses the AWS-managed key; security wants a customer-managed key. What do you do?

Answers:

1. Create `npm-store` (`public:npmjs`) and `pypi-store` (`public:pypi`), and add both as **upstreams** of the team repository.
2. **Package origin controls** - the internal package was published directly, so upstream is BLOCK.
3. Interface endpoints **codeartifact.api** and **codeartifact.repositories** (private DNS) plus an **S3 gateway endpoint** (and STS if needed).
4. **codeartifact:GetAuthorizationToken** and **sts:GetServiceBearerToken**.
5. You **can't change a domain's key** - create a new domain with the customer-managed key and migrate the packages.

3.x More exam scenarios (added in the gap review)

- **Every account in the AWS Organization, including future accounts:** in the domain and repository policies, allow Principal `*` with the condition `aws:PrincipalOrgID` = your organization ID. No updates needed when accounts are added.